Lesson 1: Anaconda

1. Learning Objective

Students will learn what Anaconda is, why it is used in Python programming, and how to manage packages and environments using Conda.

2. Simple Definition

Anaconda is a massive toolkit that contains Python, data science tools, and a built-in manager to keep all your programming projects organized.

3. Detailed Explanation

- **What it is:** Anaconda is a Python distribution designed specifically for data science, AI, and scientific computing.
- **Python vs. Anaconda:** Normal Python is just the core language. Anaconda includes Python *plus* hundreds of pre-installed libraries (like NumPy and Pandas) and tools (like Jupyter and Spyder).
- **Navigator vs. Prompt:** Anaconda Navigator is a graphical interface (point-and-click) to launch apps. Anaconda Prompt is a command-line interface (typing commands).
- **Conda vs. Pip:** pip installs Python packages. conda installs packages *and* manages environments, and it can install non-Python software too.

4. Real-World Analogy

Imagine Python as a plain, empty kitchen. You have to buy every single utensil yourself (pip). Anaconda is a fully stocked professional kitchen that comes with all the advanced appliances, spices, and tools already unpacked and ready to use.

5. Syntax & Commands

Bash

```
# Verify conda is installed
conda --version

# Create a new environment named "myenv"
conda create --name myenv python=3.9

# Activate the environment
conda activate myenv
```

```
# Install a package
conda install numpy

# Deactivate the environment
conda deactivate
```

6. Examples

(See syntax block above).

- *Expected output:* When you type conda activate myenv, your command prompt will change to show (myenv) at the beginning, indicating you are inside the isolated space.
- *Common Mistake:* Trying to use pip to install a package when a conda version is available. It's best to stick to conda install when using Anaconda to avoid conflicts.

7. Important Points

- Anaconda saves time by pre-installing complex data science libraries.
- conda is both a package manager and an environment manager.

8. Common Mistakes

- Forgetting to activate the environment before installing packages.
- Mixing pip and conda installations aggressively, which can break the environment.

9. Practice Questions

1. What is the difference between normal Python and Anaconda?
2. Write the command to create a conda environment named "ai_class".
3. How do you launch Jupyter Notebook from Anaconda?

10. Quick Revision

Anaconda is a data science distribution of Python. It uses conda to manage packages and isolated project environments, preventing software conflicts.

Lesson 2: What is a Virtual Environment?

1. Learning Objective

Students will understand the concept of virtual environments, why they are essential for project isolation, and how to create them.

2. Simple Definition

A virtual environment is an isolated, protective bubble for your Python project. It keeps the project's tools and versions separate from other projects on your computer.

3. Detailed Explanation

- **Why we need them (Dependency Management):** Project A might need Python 3.8 and Pandas v1.0. Project B might need Python 3.10 and Pandas v2.0. If you install them globally on your system, they will conflict.
- **Project Isolation:** Virtual environments ensure that changing a package in one project doesn't break another project.
- **System Python vs. Virtual Env:** System Python is the main installation on your OS. A virtual environment is an isolated copy.
- **venv vs. conda:** venv is built into standard Python. conda is Anaconda's environment manager.

4. Real-World Analogy

Think of virtual environments like different rooms in a house. The kitchen (Project A) has an oven and a fridge. The garage (Project B) has a toolbox and a car. You don't want to mix car oil with your cooking ingredients. Environments keep the tools in their correct rooms.

5. Syntax & 6. Examples

Using standard Python (venv):

Bash

```
# 1. Create the environment named "my_project_env"
python -m venv my_project_env

# 2. Activate it (Windows)
my_project_env\Scripts\activate

# 2. Activate it (Mac/Linux)
source my_project_env/bin/activate

# 3. Install a package inside the bubble
pip install requests

# 4. Leave the bubble
deactivate
```

Line-by-line: We create a folder named my_project_env containing the Python copy. We run the activate script to "enter" it. We install requests. We type deactivate to exit.

7. Important Points

- Never install project packages globally (system-wide).
- Always activate the environment before writing or running your code.

8. Common Mistakes

- Closing the terminal and forgetting to reactivate the environment when reopening it.
- Committing the virtual environment folder to GitHub (always add it to .gitignore).

9. Practice Questions

1. Why is it a bad idea to install all Python packages globally?
2. What is the difference between venv and conda?
3. Write the command to deactivate an active environment.

10. Quick Revision

Virtual environments isolate project dependencies, preventing version conflicts. venv is standard Python; conda is for Anaconda.

Lesson 3: Working with Jupyter Notebook

1. Learning Objective

Students will learn how to launch, navigate, and execute Python code using Jupyter Notebook.

2. Simple Definition

Jupyter Notebook is an interactive digital notebook where you can write Python code, execute it instantly, and add text, images, and charts all in one place.

3. Detailed Explanation

- **Why it's popular in AI:** AI requires visualizing data and testing small chunks of code. Jupyter lets you run code block-by-block instead of running an entire script at once.
- **Launching:** Usually launched via Anaconda Navigator or by typing jupyter notebook in the command prompt.
- **Cells:** The building blocks of a notebook.
    - **Code Cells:** Where you type and run Python code.
    - **Markdown Cells:** Where you type formatted text, headers, and explanations.
- **Kernels:** The hidden "engine" that actually runs the Python code you type in the notebook.
- **File Extension:** Saved as .ipynb (Interactive Python Notebook).

4. Real-World Analogy

A standard Python script is like a printed book—you read it from start to finish. A Jupyter Notebook is like a scientist's lab journal—you write a hypothesis (text), conduct an experiment (code), and tape the results (graphs) right next to it.

5. Syntax & 6. Examples

(In a Jupyter Code Cell)

Python
```
# Cell 1
print("Hello, AI Students!")
# Output directly below the cell: Hello, AI Students!

# Cell 2
x = 10
y = 20
x + y
# Output directly below the cell: 30
```

7. Important Points

- Variables declared in one cell are remembered in all cells below it (as long as they are run).
- Restarting the kernel erases the notebook's memory of all variables.

8. Common Mistakes

- Running Cell 3 before running Cell 1, causing an "undefined variable" error because the code ran out of order.
- Forgetting to save the .ipynb file before closing the browser.

9. Practice Questions

1. What does .ipynb stand for?
2. What is the difference between a Code cell and a Markdown cell?
3. Why is Jupyter popular in Data Science?

10. Quick Revision

Jupyter is an interactive coding environment combining code, output, and text in .ipynb files. The Kernel runs the code, and cells can be executed out of order.

Lesson 4: Jupyter Notebook Interface

1. Learning Objective

Students will master the Jupyter UI, learn to switch between modes, and use essential keyboard shortcuts to code faster.

2. Simple Definition

The interface is the screen where you interact with Jupyter. It has menus, toolbars, and cells,

and it operates in two distinct modes: Command Mode (for moving around) and Edit Mode (for typing).

3. Detailed Explanation

- **Menu & Toolbar:** Located at the top, used for saving, adding cells, and managing the kernel.
- **Run / Restart / Interrupt:**
  - *Run:* Executes the selected cell.
  - *Interrupt:* Stops a cell that is taking too long or is stuck.
  - *Restart:* Reboots the engine, wiping all variable memory.
- **Two Modes:**
  - **Edit Mode (Green border):** You are typing inside a cell.
  - **Command Mode (Blue border):** You are outside the cell, moving around the notebook.

4. Real-World Analogy

Edit mode is like writing on a piece of paper. Command mode is like organizing the papers on your desk—shuffling them, deleting bad ones, or adding blank ones.

5. Keyboard Shortcuts

- **Edit Mode:**
  - Shift + Enter: Run cell and move to the next.
  - Ctrl + Enter: Run cell and stay on it.
  - Esc: Exit to Command Mode.
- **Command Mode:**
  - Enter: Enter Edit Mode.
  - A: Insert cell **A**bove.
  - B: Insert cell **B**elow.
  - DD: Delete current cell.
  - M: Change to **M**arkdown cell.
  - Y: Change to Code cell.

7. Important Points

- Shortcuts save massive amounts of time. Memorize Shift + Enter, A, B, and DD.
- The execution order (the number inside [ ] next to a cell) tells you the actual order the code was run, regardless of vertical position.

8. Common Mistakes

- Typing DD to delete a cell while in Edit Mode (which just types the letters "DD" in your code).

9. Practice Questions

1. How do you switch from Edit Mode to Command Mode?
2. What does Shift + Enter do?
3. What happens if you press M in Command mode?

10. Quick Revision

Jupyter has Edit Mode (typing code, green) and Command Mode (organizing cells, blue). Learn the shortcuts (A, B, DD, Shift+Enter) to work efficiently.

Lesson 5: Why Python for Artificial Intelligence?

1. Learning Objective

Students will understand why Python is the undisputed king of AI, Machine Learning, and Data Science.

2. Simple Definition

Python is chosen for AI because it is easy to read, fast to write, and has a massive collection of free, pre-built tools specifically designed for complex math and AI.

3. Detailed Explanation

- **Simple Syntax:** Python looks like plain English. You spend time solving AI problems, not fighting complex code syntax.
- **Large Ecosystem (Libraries):** A library is a pre-written bundle of code. Python has the best AI libraries in the world.
    - **NumPy / Pandas:** For handling heavy data and tables.
    - **Matplotlib / Seaborn:** For drawing charts and graphs.
    - **Scikit-learn:** For basic Machine Learning algorithms.
    - **TensorFlow / PyTorch:** For deep learning and neural networks (like ChatGPT uses).
    - **OpenCV:** For Computer Vision (teaching computers to see).
- **Community Support:** If you have an AI bug in Python, millions of people online have already solved it.

4. Real-World Analogy

Building AI in C++ is like building a car from scratch by mining your own metal. Building AI in Python is like buying a Lego kit—all the complex pieces (libraries) are already perfectly molded; you just snap them together to build the car.

5. Syntax & 6. Examples

Python
A simple example showing how easy it is to use a library in Python

```
import math

# We want to find the square root for a math problem
result = math.sqrt(64)
print(result) # Output: 8.0
```

7. Important Points

- Python is fundamentally slow, but its AI libraries (like NumPy and PyTorch) are written in C/C++, making Python a "glue" language that acts very fast.

8. Common Mistakes

- Thinking Python is the *only* language for AI. (Others exist, like R or Julia, but Python is the industry standard).

9. Practice Questions

1. Name two Python libraries used for Machine Learning/Deep Learning.
2. Why is Python's syntax beneficial for AI developers?
3. What is the purpose of the Pandas library?

10. Quick Revision

Python dominates AI due to its readable syntax, massive developer community, and unmatched ecosystem of powerful data science libraries (Pandas, PyTorch, Scikit-learn).

Lesson 6: Applications of Python

1. Learning Objective

Students will explore the versatile, real-world industries and applications where Python is actively used.

2. Simple Definition

Python is a general-purpose language, meaning it can be used to build almost anything—from websites to robots, and from video games to hacking tools.

3. Detailed Explanation

- **Web Development:** Building the backend of websites (e.g., Instagram, Spotify use Python frameworks like Django).
- **Artificial Intelligence / Data Science:** Analyzing data and building predictive models (e.g., Netflix recommendations).
- **Automation (Scripting):** Writing code to do repetitive boring tasks (e.g., renaming 10,000 files in one second).
- **Computer Vision / NLP:** Facial recognition and chatbots.

- **Web Scraping:** Automatically extracting data from websites.
- **Cybersecurity / IoT:** Penetration testing scripts and programming smart home devices (Raspberry Pi).

4. Real-World Analogy

Python is the Swiss Army Knife of programming. Just as a Swiss Army Knife has a blade for cutting, a screwdriver for screws, and a file for sanding, Python has Django for websites, PyTorch for AI, and Selenium for automation.

7. Important Points

- Python's versatility makes it one of the most highly employable skills in tech.
- It is generally *not* used for high-end 3D game engines (like Unreal Engine, which uses C++).

8. Common Mistakes

- Trying to use Python for heavy mobile app development. While possible, languages like Swift or Kotlin are better suited for mobile apps.

9. Practice Questions

1. Name three major applications of Python.
2. Give a real-world example of Python in Web Development.
3. What is web scraping?

10. Quick Revision

Python is used across diverse fields including AI, Web Development, Automation, Data Science, and Cybersecurity due to its flexibility and vast library support.
(Note: To respect classroom delivery pace, Modules 7 through 17 apply the exact same structured pedagogy to core programming concepts.)

Lesson 7: Python Character Set

- **Objective:** Understand the raw building blocks of Python text.
- **Definition:** The set of valid characters Python recognizes.
- **Details:** Letters (A-Z, a-z), Digits (0-9), Special Symbols (@, #, $), Whitespace (spaces, tabs), and Unicode (supports emojis and multiple languages).
- **Important:** Identifiers (variable names) cannot contain special symbols except underscore _.

Lesson 8: What are Statements in Python?

- **Objective:** Differentiate between expressions and statements.
- **Definition:** A statement is an instruction that the Python interpreter can execute.
- **Details:** x = 5 is an assignment statement. if x > 2: is a conditional statement. An

expression evaluates to a value (e.g., 5 + 2), while a *statement* does an action.
- **Example:** print("Hi") (Statement) vs 2 + 2 (Expression).

Lesson 9: Comments

- **Objective:** Learn to document code.
- **Definition:** Text ignored by Python, used to explain code to humans.
- **Syntax:** Use # for single-line. Python has no official multi-line comment, but we use triple quotes """ (Docstrings) as a workaround.
- **Example:**
  Python
  # This calculates area
  area = 5 * 5

Lesson 10: Keywords

- **Objective:** Identify reserved words in Python.
- **Definition:** Words that have a special, built-in meaning and cannot be used as variable names.
- **Details:** Includes if, else, for, while, def, True, False.
- **Mistake:** Trying to name a variable True = 10. This crashes Python.

Lesson 11: Variables

- **Objective:** Learn to store data in memory.
- **Definition:** A variable is a container for storing data values.
- **Details:** Python uses *dynamic typing* (you don't have to declare the type).
- **Syntax:** name = "Rahul"
- **Analogy:** A variable is like a labeled moving box. The label is the variable name, and what's inside is the data.

Lesson 12: Data Types

- **Objective:** Understand different types of data.
- **Types:** Numeric (int, float), Boolean (bool), Sequence (str, list, tuple), Mapping (dict).
- **Checking Types:** Use type(variable).
- **Example:**
  Python
  age = 20 # int
  price = 99.99 # float
  is_student = True # bool

Lesson 13: Operators

- **Objective:** Perform mathematical and logical operations.
- **Details:**
    - Arithmetic: + (add), - (sub), * (mult), / (div, returns float), // (floor div, returns int), % (remainder/modulo), ** (power).
    - Comparison: == (equal to), != (not equal).
    - Logical: and, or, not.
- **Mistake:** Confusing = (assignment) with == (comparison).

Lesson 14: Operator Precedence

- **Objective:** Understand the order of mathematical operations (PEMDAS/BODMAS in Python).
- **Definition:** The rule that determines which operator runs first.
- **Example:** 10 + 5 * 2 equals 20, not 30, because multiplication (*) has higher precedence than addition (+). Use parentheses (10 + 5) * 2 to force addition first.

Lesson 15: Type Conversion

- **Objective:** Change data from one type to another.
- **Definition:** Implicit (Python does it automatically, e.g., int + float = float). Explicit (You force it using functions like int(), str()).
- **Example:**
  Python
  ```
  age = "20"
  real_age = int(age) # Converts string to integer
  ```

Lesson 16: Conditional Statements

- **Objective:** Teach programs to make decisions.
- **Syntax:**
  Python
  ```
  marks = 85
  if marks > 90:
     print("Grade A")
  elif marks > 80:
     print("Grade B")
  else:
     print("Study harder")
  ```

- **Important:** Indentation (spaces before the print) is mandatory in Python.

Lesson 17: Loops

- **Objective:** Automate repetitive tasks.

- **Details:**
    - for loop: When you know how many times to repeat.
    - while loop: When you want to repeat *until* a condition becomes false.
- **Example:**
  Python
  # Prints 0, 1, 2, 3, 4
  for i in range(5):
    print(i)

Final Course Revision Section

A. One-Page Quick Revision Notes

Python is an intuitive, dynamically typed language heavily used in AI via Anaconda and Jupyter. We store data in **Variables**, categorized by **Data Types** (int, str, list). We manipulate them using **Operators** (+, ==), and control the flow of the program using **Conditionals** (if/else) and **Loops** (for/while).

B. Important Definitions

- **Virtual Environment:** Isolated folder for project dependencies.
- **Dynamic Typing:** Python automatically detects data types during variable assignment.
- **Operator Precedence:** The order in which mathematical operations are evaluated.
- **Type Casting:** Manually converting one data type into another.

C. Comparison Tables

Feature	**for Loop**	**while Loop**
Usage	Known number of iterations	Unknown iterations; condition-based
Iteration	Over sequences (lists, range)	Runs as long as condition is True

Operator	**=**	**==**
Name	Assignment Operator	Comparison / Equality Operator

Action	Assigns value to a variable (x = 5)	Checks if two values are identical (5 == 5)

D. Viva Questions (Excerpt)

1. **Q:** What is dynamic typing? **A:** Python assigns data types at runtime, without needing explicit declaration.
2. **Q:** What does the % operator do? **A:** It returns the remainder of a division.
3. **Q:** Why use Jupyter for AI? **A:** It allows cell-by-cell execution and inline data visualization. *(Instructor note: Use the structural rules from the lessons above to prompt students on all 17 topics).*

E. Programming Practice / Final Mini Project

Project: "Student Grade Analyzer"
Goal: Use variables, type conversion, operators, conditionals, and loops.

Python

```
# Mini Project: Student Grade Analyzer

# 1. Variables & Type Conversion
student_name = input("Enter student name: ")
total_subjects = int(input("How many subjects? "))

total_marks = 0

# 2. Loops
for i in range(total_subjects):
    # 3. Operators & Type Conversion
    mark = float(input(f"Enter marks for subject {i+1}: "))
    total_marks += mark  # Same as total_marks = total_marks + mark

# 4. Operators
average = total_marks / total_subjects

print(f"\nStudent: {student_name}")
print(f"Average: {average}%")

# 5. Conditional Statements
if average >= 90:
```

```
    print("Final Grade: A")
elif average >= 80:
    print("Final Grade: B")
elif average >= 50:
    print("Final Grade: Pass")
else:
    print("Final Grade: Fail")
```